

课程笔记

MIT RES.9-009 课程讲义第 01 讲: Intro to Neuroblox

MIT RES.9-009 课程整理 & Codex

2026 年 4 月 15 日



视频作者/频道: MIT Video Productions

发布日期: 2025-01-13

视频时长: 57:16

视频链接: <https://www.youtube.com/watch?v=2dbAePEmbhQ>

目录

1	课程开场、工作方式与整体路线	2
1.1	本章小结	2
2	什么是计算建模：模型要回答什么问题	2
2.1	本章小结	3
3	Neuroblox 的多尺度科学问题	3
3.1	本章小结	4
4	从相关性到动力系统：为什么要模拟	4
4.1	本章小结	4
5	GUI、平坦大脑与模块化 workflow	5
5.1	本章小结	6
6	为什么是 Julia：性能、可组合性与平台基础	6
6.1	本章小结	7
7	总结与延伸	7
7.1	拓展阅读	7
7.2	本章小结	7

1 课程开场、工作方式与整体路线

这门课一开始就把两个事实说得很清楚。第一，课程本身是一个 IAP 里的短时密集工作坊，讲者希望把大量时间留给你在 notebook 里动手。第二，Neuroblox 还是一个正在快速迭代的系统，视频里的讲者甚至明确提醒你们是第一批非开发者用户之一，因此遇到 bug 不要惊讶，而要把它当作课程流程的一部分。

从官方课程页和 MIT OCW 页面看，这门课的定位也完全一致：它强调 hands-on model building in Neuroblox and Julia，强调从 literature 中的模型出发，最后把模型拟合到神经数据，并进一步探索 drugs 和 stimulation probes 这类干预。换句话说，视频开场讲的“先给背景，再做练习”，并不是随口一说，而是官方课程结构本身的一部分。

为什么一开始就要报告 bug

讲者不是在推卸责任，而是在定义这门课的协作方式。Neuroblox 仍处于 beta 阶段，很多问题会出现在真实使用场景里，而这些场景恰好是开发者事先很难完全预料到的。换句话说，学生的“卡住”本身就是产品成熟化的重要反馈信号。

从教学组织上看，这门课明显不是“先讲完理论，再做一次大作业”的传统结构，而是“每次先给一个 10 到 15 分钟的概念框架，然后把剩余时间留给 notebook 和练习”。这一点非常重要，因为 Neuroblox 的目标并不是只让你理解术语，而是让你学会在同一套生态里把模型、参数、连接规则、仿真和分析串起来。

这门课的路线也很清楚：先完成安装和环境准备，再进入 Julia，接着学习微分方程与绘图，然后进入 Neuroblox 的 blocks、连接、神经元、神经质量、回路、决策、突触可塑性、参数拟合、谱 DCM 和实验设计。你可以把它理解为一从“能跑起来”到“能搭系统”再到“能做推断和设计”的路径。

整门课的组织逻辑

课程不是按“软件功能列表”展开，而是按“科学建模能力”展开。最前面解决环境和语言问题，中间解决单个模块和单个回路问题，后面再解决模型比较、拟合和实验设计问题。这个顺序本身就体现了 Neuroblox 的目标：它不是单纯的画图工具，而是一套面向计算神经科学研究的建模语言与 workflow。

1.1 本章小结

本讲的开场重点不在技术细节，而在工作方式：Neuroblox 还是 beta 软件，课程鼓励真实使用、真实报错和真实修复；同时，后续所有内容都围绕 notebook 实操展开，而不是只停留在概念演示。

2 什么是计算建模：模型要回答什么问题

讲者用一个很重要的观点切入：模型的好坏，不在于它是不是“把所有细节都装进去”，而在于它是否与研究问题匹配。这个意思并不抽象。若研究目标是解释所有可能的现象，那么模型就必须接近系统本身；但现实研究里，我们几乎总是要对分辨率做战略选择。

这也是为什么讲者借“Lemma the cat”的玩笑来说明模型哲学。一个模型如果想把对象的每个特征都一一复刻，就会迅速滑向过拟合，最后反而失去可解释性。更好的做法是先问：我要回答的是哪类问题，哪些变量是必要的，哪些细节可以暂时抽象掉。

模型分辨率是设计选择，不是默认值

如果你的问题是“为什么这个病程会沿着某条轨迹演化”，那你需要时间和动力系统。若你的问题是“这个系统在这个尺度上由哪些部件组成”，那你需要结构化的组件和连接。Neuroblox 选择的不是某一种“唯一正确”的分辨率，而是允许你在不同尺度上构造模型，并让这些尺度彼此对接。

这直接引出 Neuroblox 的定位。讲者明确把它描述成一个面向生物医学工程的工具，关注的是“机制整合”和“治疗相关靶点”。它并不只是为了复现某个漂亮的神经科学图，而是要让你能把病理机制、药物作用、刺激装置和行为输出放在同一个建模框架里讨论。

Neuroblox 关注的问题也很具体。比如痴呆、抑郁、成瘾、双相障碍、精神病性障碍都不是简单的二元标签，而是有轨迹、有阶段、有退化、有起病时间差异的动态过程。讲者反复强调，临床表型的差异并不意味着底层回路一定不同，很多时候只是同一回路以不同方式失调。

- 痴呆的问题不是“有或没有”，而是“如何进展、谁有风险、何时加速”。
- 双相障碍更像状态切换问题，而不是静态分类问题。
- 成瘾与抑郁也常呈现长期轨迹，而不是单次测量即可解释的离散标签。

2.1 本章小结

好的模型不是“越细越好”，而是“分辨率与问题匹配”。Neuroblox 的设计哲学正是围绕这个原则展开的：让模型既能跨尺度，又能保持研究问题所需的可解释性。

3 Neuroblox 的多尺度科学问题

从课程和讲者的描述看，Neuroblox 要处理的是一个跨度很大的尺度链条。最低层可以是离子通道和受体，中间层可以是神经元、神经质量和脑区，更高层可以是回路、脑区间交互、神经影像信号，最终输出则可能是任务表现、情绪状态或者临床症状。

尺度层级	典型对象	典型问题
分子/受体层	离子通道、受体、代谢	药物怎样改变底层动力学
细胞层	神经元、放电模式	参数变化如何改变尖峰样式
中尺度	神经质量、脑区	振荡与平均活动如何演化
回路层	多脑区网络	DBS 或 TMS 会怎样影响回路
行为层	任务、情绪、症状	模型能否再现临床轨迹

讲者给出的例子非常一致。对于药物，问题不仅是它影响神经递质水平，还要看受体动力学、连接规则和更高层回路如何一起变化。对于代谢，问题不仅是“能量代谢异常是否存在”，而是“代谢异常通过什么路径影响脑功能”。对于刺激，问题不仅是“刺激是否有效”，而是“刺激如何沿着回路反馈回来，并在行为上留下轨迹”。

在这里，Neuroblox 的独特之处不是“能不能模拟一个单独模块”，而是“能不能把模块拼成一个可解释的多尺度系统”。讲者提到的 neurodiagnostic、pharma module、metabolic module 和 neuromodulation module，都是围绕这一目标展开的扩展方向。

多尺度建模的核心不是堆叠，而是贯通

如果一个回路模型只能给出静态连接图，那它很难回答“为什么这个患者在两周后恶化”这样的问题。Neuroblox 要做的是让代谢、刺激、连接、受体和行为都进入同一条时间轴上，然后观察系统如何在反馈中演化。

3.1 本章小结

Neuroblox 要解决的是多尺度、动态、可干预的问题。它把分子、细胞、脑区、回路和行为放到同一个框架里，目标不是分类而是模拟，不是静态描述而是轨迹解释。

4 从相关性到动力系统：为什么要模拟

讲者在这一讲里花了很多时间区分两类“看起来像模型”的东西。第一类是 neuroimaging 里常见的 connectivity、GLM、ICA、meta-analysis 这类统计方法，它们可以帮助你找出相关模式、提高可靠性、做 feature extraction 或 decoding。第二类才是 Neuroblox 关注的 computational neuroscience：一个能够被真正仿真的动力系统。

这两类方法各有用处，但任务不同。统计方法适合问“哪些区域共同激活”“哪些特征能区分状态”“某个模式是否可重复出现”；动力系统则适合问“这个系统在时间上如何演化”“一个药物或刺激会怎样改变轨迹”“多级反馈会不会导致状态翻转或崩溃”。

讲者特别强调，相关性方法在有反馈的系统里会非常脆弱。因为当你把多个 pairwise correlation 一层层串起来时，误差会乘法式地放大。即使单条边的相关系数看起来不错，一旦进入多边网络，整体解释力也会迅速下降。换句话说，“谁和谁相关”并不能自动还原“系统如何工作”。

这也是为什么讲者用 glucose-insulin regulation 作为一个很好的 ODE 类比。你吃一个糖块，系统先响应 glucose，再响应 insulin，再把 glucose 拉回去；如果调节失衡，你会看到不同的长期轨迹，甚至是 overshoot and collapse。这个例子非常适合说明：临床问题往往是时间尺度很长的动力学问题，而不是一次性测量可以解决的静态分类问题。

可以把讲者在这里的意思抽象成一个最一般的状态方程：

$$\dot{x}(t) = f(x(t), u(t); \theta)$$

其中， $x(t)$ 表示系统在时刻 t 的状态， $u(t)$ 表示外部输入或干预， θ 表示参数， f 表示决定系统如何随时间演化的动力学规则。这里的写法不是在替代讲者的具体例子，而是把“输入、状态、参数和时间演化”这几个核心关系抽象出来，方便你把 glucose-insulin regulation、药物作用和刺激响应放进同一类动力系统里理解。

为什么仿真比相关性更适合回答临床问题

临床和实验室的观察窗口很短，但疾病演化可能持续数月、数年甚至数十年。只有把系统写成能时间推进的动力系统，你才可能讨论慢变量、长期轨迹、状态翻转和治疗后的反馈效应。

这一讲里还有一个重要的观点：Neuroblox 不是要完全替代统计方法，而是要把问题推进到“机制可模拟”的层面。对于神经影像语境中的网络、回路和 computational neuroscience，讲者提醒你要区分两件事：一件是用统计工具提取模式，另一件是构造可解释、可干预的动态模型。前者回答“像什么”，后者回答“为什么会这样、如果我干预会怎样”。

4.1 本章小结

相关性分析擅长描述模式，动力系统擅长解释轨迹。Neuroblox 站在后者一边，因为它要处理药物、刺激、代谢和行为的时间演化，而不是只给出静态标签。

5 GUI、平坦大脑与模块化 workflow

讲者把 Neuroblox 的核心界面讲得很具体：它有一个 flat brain，也就是把整张脑的结构展开成平面地图，让你把 block 放到对应的 brain region 上；它有节点和边、参数和 simulation 等侧栏；它还允许你通过 Alt 或 Option 方式建立连接、通过拖拽改变边的几何形状，并在同一界面里运行仿真、观察结果和修改参数。

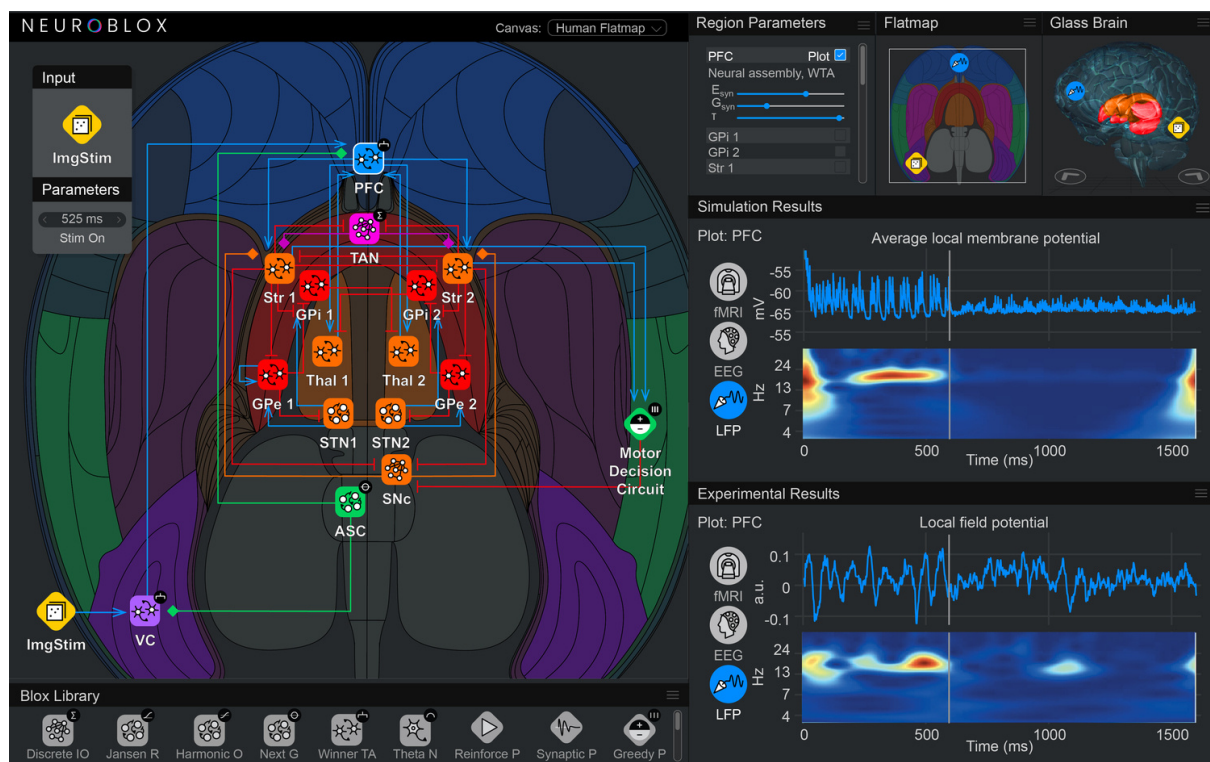


图 1: Neuroblox GUI 截图。画面中同时展示了 flat brain 视图、节点布局、仿真结果和实验结果面板，能直观体现 Neuroblox 的“回路 + 地理 + 输出” workflow。¹

这张图的价值在于，它把抽象的模块化思想具体化了。Neuroblox 不是只让你写一串方程，而是让你把方程封装成 block，再把 block 放到脑区上，最后把整个系统当成一个图来编辑。讲者甚至强调，系统的层次关系是“blocks of blocks of blocks”。这不是口号，而是整个平台的设计原则。

这种设计的关键优点是可复用和可比较。若不同团队都用同一套模块定义方式，那么模型之间就更容易对比、整合和扩展，而不是每个组都用自己的术语、自己的接口、自己的连接规则。讲者的意思可以归纳为：Neuroblox 不是单个模型，而是一个方便把模型统一管理、统一比较的建模环境。

更重要的是，Neuroblox 的模块不是空壳。讲者举了几个非常具体的例子：lateral inhibition、winner-takes-all circuits、cortical region、pharma module、metabolic module、neuromodulation module。每个模块都对应一种真实的神经科学功能假设，而不是为了视觉上好看而做的装饰。

¹ 视频画面时间区间：00:33:45–00:36:10。

- ‘nodes and edges’ 用来编辑局部结构，节点对应 block，边对应连接规则。
- ‘parameters’ 面板允许你把许多连接统一到一个共享参数上，便于整体调参。
- ‘simulation’ 面板允许你设定时长、容差、求解器和时间步进策略。
- 右侧输出面板让你把模拟结果和实验结果并排比较，便于直接检视模型是否抓住了数据里最重要的动态特征。

讲者还强调了一个很现实的问题：GUI 是给更多人用的，而不是只给“住在洞穴里的硬核计算神经科学家”用的。实验人员、理论人员、临床方向的人都应该能够理解和使用它。所以 GUI 的设计既要保留神经科学直觉，也要支持复杂的模拟和数据分析。

平坦大脑为什么有意义

讲者采用 Mercator projection 的类比，说明为什么要把脑展开成 flat map。目的不是做一个好看的背景，而是让你在一个视图里同时看到 cortical、subcortical、front/back 等不同区域，并把机制图层、输入、输出和区域地理叠加起来。

讲者最后还提醒，Julia 是编译型语言，所以第一次运行时往往会先付出编译开销；但一旦编译完成，后续运行就会快得多。长期看，JuliaHub 这类云平台会成为重要的托管方式。

5.1 本章小结

Neuroblox 的 GUI 不是附属品，而是平台的一部分。它把模块、连接、地理位置、参数和输出统一到 flat brain 上，使研究者既能沿着神经科学直觉建模，又能沿着软件工程原则复用和比较模型。

6 为什么是 Julia：性能、可组合性与平台基础

讲座后半段给出了选择 Julia 的核心原因。第一是所谓 two-language problem。高性能科学计算通常会把“开发快”和“运行快”分成两套语言来做，例如用 Python 组织逻辑、再把性能关键部分交给 C 或 C++。讲者认为 Julia 的价值就在于把这两件事统一起来：它像 Python 一样易写，又能像 C 一样跑得快。

这种性能并不是空口说白话，而是和 Julia 的编译模型有关。讲者强调 Julia 是 compiled language，并提醒第一次运行会因为编译而较慢，但后续会快很多。更关键的是，Julia 的 solver 生态非常成熟，尤其在 ODE、SDE、DAE 以及延迟微分方程等场景下都很强。这对 Neuroblox 这种大量依赖动力系统仿真的平台非常重要。

Julia 的第二个优势是自动微分 (automatic differentiation)。讲者特别强调，它之所以关键，是因为 Neuroblox 需要 parameter fitting、optimization 和 Bayesian posterior estimation。换句话说，只有模型可微，参数估计、模型比较和逆问题求解才真正可做。

为什么可微性会影响研究范式

可微不只是“能不能算梯度”的技术细节，而是决定你能否从“手工调参”走向“自动拟合”。对于 Neuroblox 来说，这意味着模型不仅能跑，还能拿真实数据来拟合、比较和排序。

Julia 的第三个优势是 composability，也就是可组合性。讲者用一类方程系统的连接过程来说明：你可以把一个系统写成方程，再把两个 block 的输出和输入连起来，最后把整个连接关系整理成可以求解的 ODE system。这个过程和 Neuroblox 的 block 设计几乎是同构的。

更进一步，讲者解释了为什么大型系统会推动 Graph Dynamics 这类库的发展。因为当系统非常大时，直接做符号化简会很慢，编译时间甚至会超过运行时间。Graph Dynamics 把系统视为 graph，用节点和边来表达连接规则，从而大幅提高大型图系统的编译效率。这一点对 Neuroblox 尤其重要，因为它面对的就是大规模、层级化、可复用的神经系统图。

最后，讲者还提到了 Julia 社区、文档和学习资源。对一个快速演化的平台来说，生态并不是附属品，而是生产力的一部分。Neuroblox 之所以能把课程、GUI、动力系统、拟合和优化连接在一起，背后依赖的正是 Julia 的语言设计、相关数值工具链和可组合建模方式。

6.1 本章小结

Julia 不是仅仅“因为快”才被选中，而是因为它同时满足三件事：交互式开发、数值性能和可组合建模。对 Neuroblox 来说，这三点分别对应 notebook 开发、ODE 求解和 block 级别的系统组装。

7 总结与延伸

这一讲真正建立起来的不是某个具体模型，而是一套建模观。Neuroblox 的目标是把神经科学里常见的机制单元标准化成可组合 block，再用 Julia 的性能和可微性把这些 block 变成可仿真、可拟合、可比较的模型系统。它关心的不是“有没有一个漂亮的静态图”，而是“是否能把机制、干预和行为放在同一条时间轴上解释清楚”。

从研究方法上看，这一讲明确区分了统计相关和机制模拟。前者适合做模式发现、可重复性分析和 decoding；后者适合做干预、长期轨迹和状态演化。Neuroblox 选择的是后者，但并不排斥前者，而是试图把前者的经验转化为后者可使用的结构化输入。

从软件工程上看，这一讲也很清楚地告诉你，Neuroblox 的核心不是“库里有多少函数”，而是“有没有一套共通的模块语义”。只要模块语义统一，社区模型就能被整合、比较、复用和扩展；如果模块语义不统一，再多的模型也只是彼此孤立的艺术品。

后续学习应该抓住的三个主线

第一，先把 Julia 和 notebook 工作流跑通。第二，理解 differential equations、block 连接和 plotting 这三者如何衔接。第三，再回到 Neuroblox 的 block、连接、层级命名和 GUI，理解为什么这个平台能把模型组装成研究系统。

7.1 拓展阅读

如果你想继续顺着这一讲往下走，最值得先看的官方材料是课程主页、MIT OCW 页面，以及后续单元里的 Getting Started、Introduction to Julia、Differential Equations in Julia 和 Plotting with Makie。它们共同组成了这门课的“语言层”和“平台层”。

7.2 本章小结

这一讲的核心结论可以浓缩成一句话：Neuroblox 试图把神经科学中的机制、尺度、干预和比较统一为一个可组合、可微分、可仿真的系统，而 Julia 正是承载这一目标的语言与生态。